

Custom Properties and Nesting Report

Parker Woolsey

2 May 2026

Site Details

I have completed my [ward activity board](#) page and updated my [GitHub repository](#).

Reflections

You wrote all of your CSS using native nesting. Compare the experience to writing flat, non-nested CSS. Where did nesting make your stylesheet clearer, and was there a point where it started to feel like too much? How would you decide where to stop nesting in a larger project?

Using native CSS nesting felt much more organized compared to flat CSS. Instead of repeating long selectors, I could group related styles together, which made it easier to read and understand the structure of my page. For example, keeping all `.card` styles and its child elements nested in one place made it clear how everything connects. It also reduced duplication and made edits faster.

However, I did notice that too much nesting can become overwhelming. If you go too deep, it starts to feel harder to scan and can actually reduce clarity. In a larger project, I would limit nesting to about 2–3 levels deep and only use it where elements are closely related. This keeps the code readable while still getting the benefits of structure and organization.

Your stylesheet derives values from tokens using `calc()` and `clamp()`. Pick one of those derived values and explain what would happen if you changed the base token it depends on. How many places in your stylesheet would be affected, and why does that matter on a team?

One example of a derived value in my stylesheet was using `calc()` to create spacing between cards based on a base spacing token. If I changed that base spacing value, it would automatically update every place that depended on it, including gaps, padding, and margins.

This matters a lot in a team setting because it creates consistency across the entire design. Instead of manually updating multiple values in different places, I only need to change one token. This reduces errors and saves time, especially as a project grows. It also makes it easier for teammates to understand the design system and maintain it.

Look at your finished ward activity board at a narrow mobile width and a wide desktop width. Describe one specific thing that `clamp()` or your responsive layout handled automatically that would have required a media query in traditional CSS:

One thing that `clamp()` handled really well was the page padding and heading sizes. On smaller screens, the padding shrinks so the content fits nicely without feeling cramped, and on larger screens it expands to create more breathing room. The same thing happens with font sizes, where headings scale smoothly instead of jumping between fixed sizes.

Without `clamp()`, I would have needed multiple media queries to adjust these values at different breakpoints. Using `clamp()` made the design more fluid and responsive with much less code, which keeps the stylesheet simpler and easier to maintain.